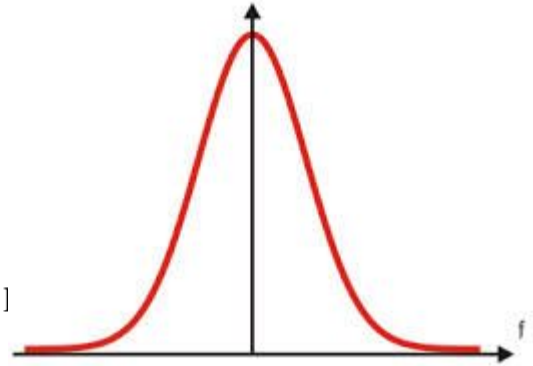
Task 1.1: Gaussian modelling

- **1 Gaussian function to model each background pixel**
  - First 25% of the test sequence to model background
  - Mean and variance of pixels

for all pixels i **do**
  if $|I_i - \mu_i| \geq \alpha \cdot (\sigma_i + 2)$ **then** ▷ +2 to prevent l
    pixel → Foreground
  else
    pixel → Background
  end if
end for



- **Second 75% to segment the foreground**
  - Group into objects → Bounding Boxes

Task 1.1: Gaussian modelling (Team 5)

Implementation

We model **each pixel independently** as a static normal distribution: $X \sim N(\mu, \sigma^2)$

- **μ (mean)** → Expected background intensity of the pixel.
- **σ² (variance)** → Natural variation of that pixel over time.

For Week 1 the model is initialized using the **first 25% of the video frames**, assuming they mostly contain background. For each pixel:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i \qquad \sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$$

A pixel is classified as **foreground** if:

$$|I_i - \mu_i| \geq \alpha \cdot (\sigma_i + 2)$$

EFFICIENCY

Since videos are large, we **do not load the entire video into memory**.

Instead: We read the video **frame by frame.**

- For computing μ and σ:
  - We **accumulate pixel sums incrementally.**
  - After processing the required frames, we divide by the total number of frames.

Post Processing / Morphology

Raw foreground masks are noisy so post-processing is essential.

Morphological Operations

We use a 5×5 elliptical kernel:

- **Opening** → Removes small noise blobs.
- **Closing** → Fills small holes inside detected objects.

This significantly improves mask quality.

After segmentation, some objects appear fragmented so we merge nearby bounding boxes based on **perspective-scaled distance**:

- Objects farther in the scene appear smaller.
- Distance thresholds are scaled according to perspective.

Task 1.1: Gaussian modelling (Team 5)

Full Pipeline

1 Input

We load the video using TorchCodec and process it frame by frame.
Each frame is transferred to the GPU for efficient pixel computation.

2 Gray Scale

Before modeling the background, we convert each RGB frame to grayscale using the standard luminance formula:

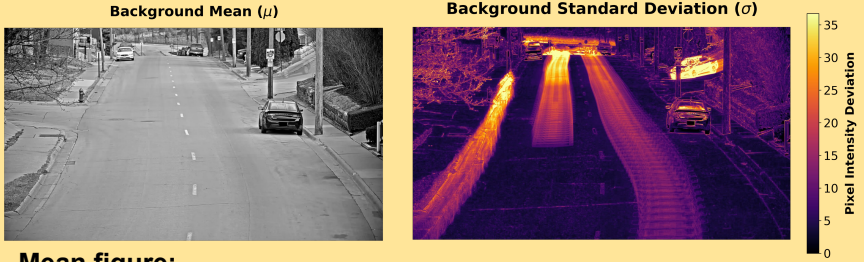
$$I = 0.299R + 0.587G + 0.114B$$

This simplifies the problem to one intensity value per pixel, reduces computation, and stabilizes the background statistics.

3 Gaussian Detection

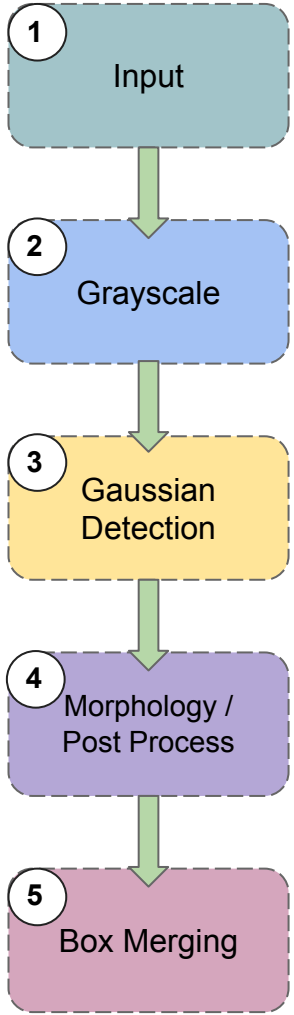
Training Phase: We use the first portion of the video to estimate a per-pixel Gaussian background model: Compute the mean intensity per pixel and compute the standard deviation per pixel This represents the expected background appearance.

Detection Phase: For each new frame, a pixel is classified as foreground given the formula from previous slide



Mean figure:
Represents the expected background intensity. Moving objects are removed via temporal filtering.
Std. figure:

- Blue/Black: Stable regions (Road). High sensitivity to detections.
- Yellow/Red: Noisy regions (Cars/Tree leaves). Low sensitivity to prevent False Positives.



Task 1.1: Gaussian modelling (Team 5)

Full Pipeline

4 Morphology / Post Processing

The raw foreground mask is often noisy. We apply morphological operations to:

- Remove small isolated pixels
- Fill small holes
- Smooth object regions

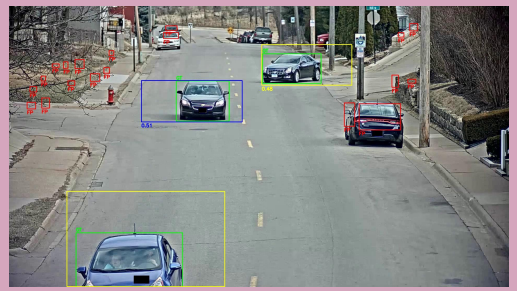
Morphological operations used were opening and closing with a 5x5 ellipse kernel.

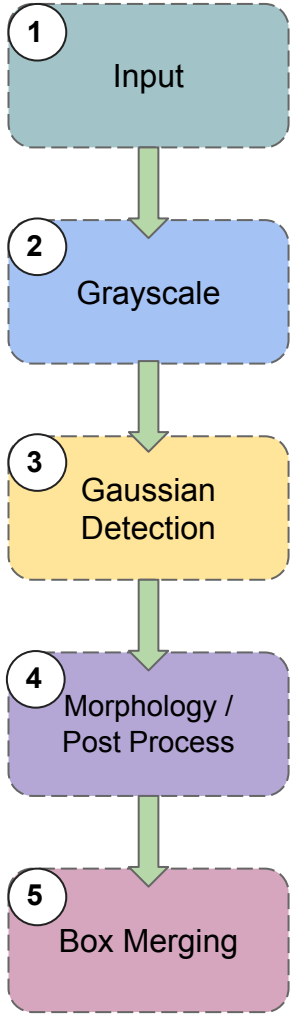
Then we extract connected components and filter out very small regions.

5 Box Merging

Finally, we generate bounding boxes around detected regions with cv2.connectedComponentsWithStats which takes a binary image (foreground mask) and finds groups of connected white pixels.

Overlapping fragmented regions are merged to ensure each vehicle is represented by a single bounding box.

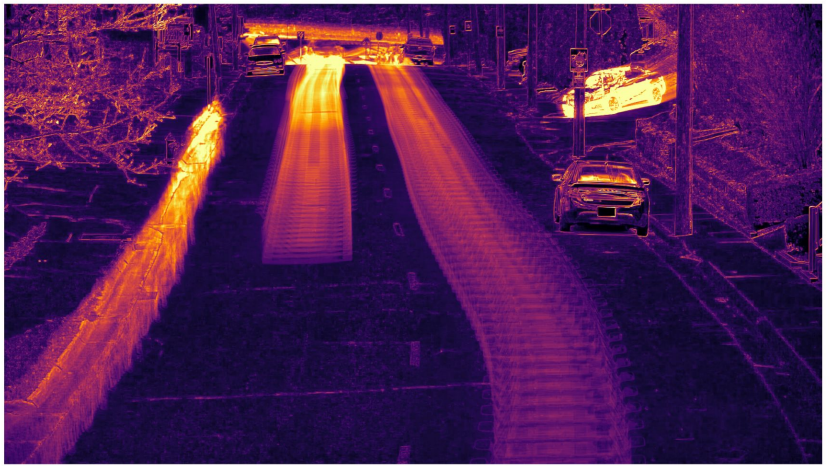




Task 1.1: Gaussian modelling (Team 5)

Background Mean (μ)

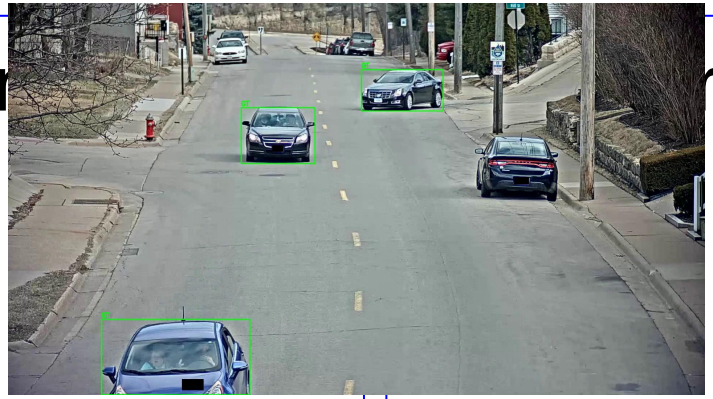
Background Standard Deviation (σ)



Yo aquesta i la 6 les treuria

Task 1.1: Gaussian modelling (Team 5)

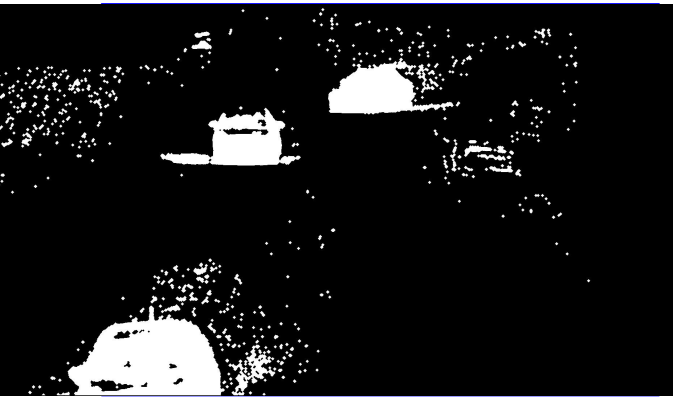
Origi

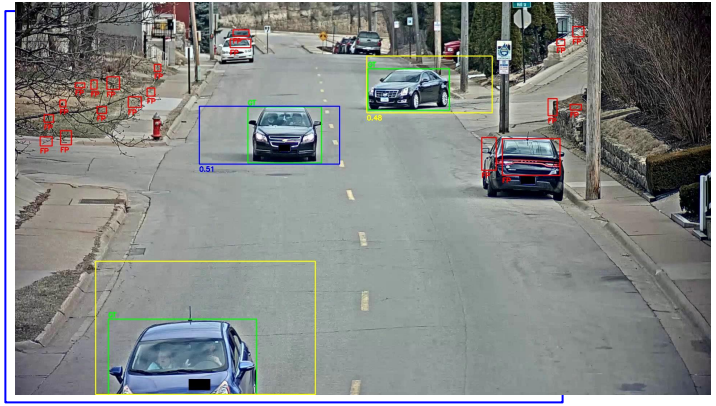


ruth

Comment

Yo aquesta i la 5 les treuria





Task 1.2: Evaluation

- **Evaluate Task 1**
  - mAP on detected connected components
  - Filter noise and group in objects
  - Over alpha threshold
  - Decide (and explain) if parked/static cars are considered

Task 1.2: mAP vs alpha (Team 5)

Experiment

We performed a grid search in order to find the optimal value of alpha for our model setup.

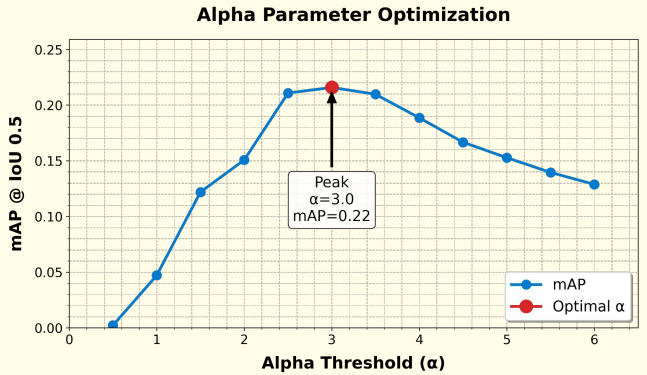
Alpha

Determines how many standard deviations a pixel's color must be away from the background mean to be considered "foreground".

Results

As we increase α, we observe that performance initially improves. The model reaches its peak performance at α = 3.0, where the mAP is approximately 0.22, computed using pycocotools to ensure standardized COCO-style evaluation.

However, beyond this point, performance starts to decline again. This indicates that the model is highly sensitive to the α parameter: if α is too small or too large, performance drops noticeably.



Experiment conclusion

From this analysis, we can conclude that proper tuning of α is critical. The model strongly depends on selecting the right threshold, and even small deviations from the optimal value can significantly reduce overall performance.

Task 1.2: mAP vs alpha (Team 5)

Limitations

False Positive scenario

We can see here a false positive. The model incorrectly detects a car due to sky illumination reflected on the windshield. This shows how the model is reacting strongly to brightness or reflection changes rather than true object structure.

Over-segmentation

The model detects two separate moving regions instead of one full car. Instead of recognizing the car as a single object, it fragments the detection into parts. This indicates weak spatial or structural understanding.

Conclusions

These examples highlight an important point: Even with optimal alpha tuning, the model still produces systematic visual errors. So we can assure that:

Tuning a single parameter is not enough.

Adjusting α may improve mAP slightly, but it does not fix these fundamental detection issues.

To address these errors, we likely need architectural improvements, and some more parameters to work with.

Task 2.1: Adaptive modelling

- **Adaptive modelling**
  - First 25% frames for training
  - Second 75% left background adapts

$$
\begin{aligned}
&\textbf{if } \text{pixel } i \in \text{Background } \textbf{then} \\
&\quad \mu_i = \rho \cdot I_i + (1-\rho) \cdot \mu_i \\
&\quad \sigma_i^2 = \rho \cdot (I_i - \mu_i)^2 + (1-\rho) \cdot \sigma_i^2 \\
&\textbf{end if}
\end{aligned}
$$

- **Best pair of values (α, ρ) to maximize mAP**
  - Two methods:
    - Obtain first the best α for non-recursive, and later estimate ρ for the recursive cases
    - Optimize (α, ρ) together with grid search or random search (discuss which is best…).

Task 2.1: Adaptive modelling (Team 5)

Adaptive update → **HSV Shadow removal** → **Buffering update** → **Morphology**

Base concept

When the background changes, we update our background using the following update of the mean and std.

$$\mu_t = (1-\rho)\mu_{t-1} + \rho I_t$$
$$\sigma_t^2 = (1-\rho)\sigma_{t-1}^2 + \rho(I_t - \mu_t)^2$$

With **rho** (ρ) we determine how many pixels to use as a **window to update the mean and std.**

HSV Shadow removal

We used the Cucchiara-R method [1] to eliminate the shadows.
Condition 1) identifies the darkening of an image pixel while ensuring that it has not become completely black relative to the background.
Conditions 2) and 3) ensure that the saturation and hue of the pixel haven't changed significantly relative to the background.

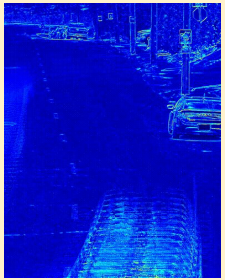
If the three conditions are satisfied we consider the pixel a shadow.

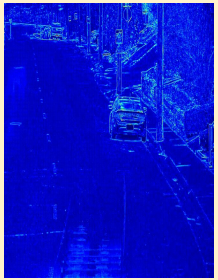
1) $\alpha \leq \frac{I_t^V}{B_t^V} \leq \beta$
2) $|I_t^S - B_t^S| \leq \tau_S$
3) $|I_t^H - B_t^H| \leq \tau_H$

Buffering update

Slow-moving cars and bikes sometimes cause the model to identify the edge of the object as the background. When the object moves forward, the car/bike's solid body is on the learned pixel, which has the same color as the object itself. This leads to a fragmented edge.

To fix this, we create an n-pixel area around each object (dilation) and update the background accordingly.


Std w/o buffering


Std with buffering


Before buffering

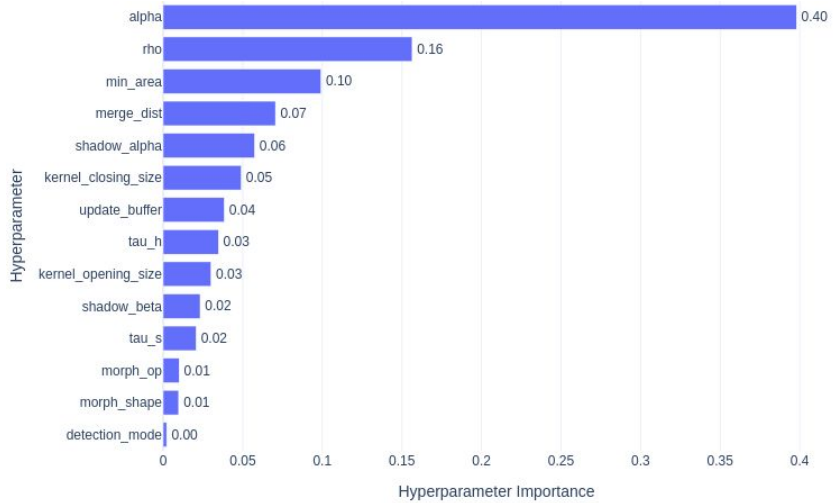

After buffering
(area not updatable)

Morphology as params

We treated the different morphology operations as parameters in order to optimize them.
Operations: Open, Close, Open & Close, Close & Open
Kernels: Rectangle, Ellipse

[1] Cucchiara, R., Grana, C., Piccardi, M., & Prati, A. (2003). *Detecting moving objects, ghosts, and shadows in video streams.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 25(10), 1337-1342.

Task 2.1: Adaptive modelling (Team 5)



The Bayesian search identified an effective strategy based on two key ideas:

1. **Use a relatively large α value:** A high α aggressively suppresses noise and small fluctuations. This removes many false positives caused by reflections and illumination changes. However, it may also erode parts of some vehicles.

2. **Apply only one morphological closing operation with a large kernel.** The large closing reconnects fragmented regions. It helps merge separated car parts into a single coherent detection. No additional morphological operations are applied.

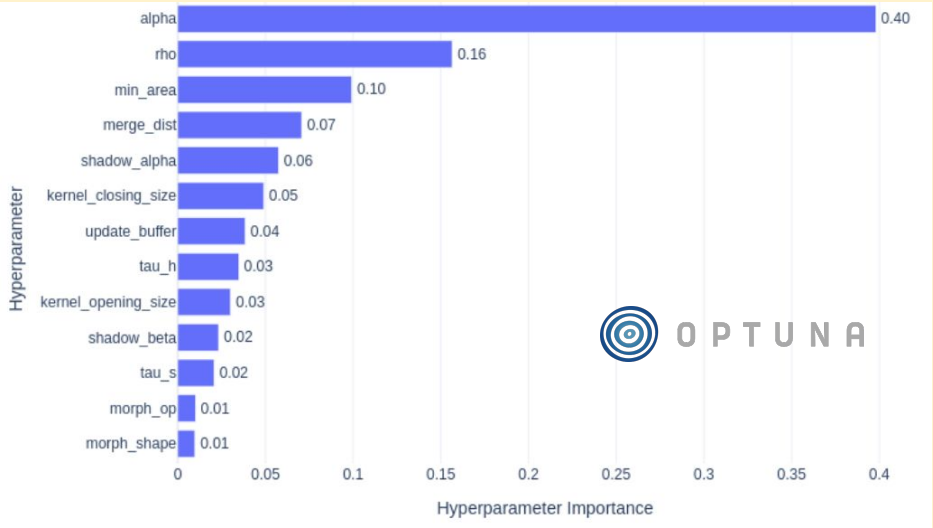
Task 2.1: Adaptive modelling (Team 5)

Bayesian search

With the Bayesian search we identified an effective strategy based on two key ideas:

1. **Use a relatively large α value:** A high α aggressively suppresses noise and small fluctuations. This removes many false positives caused by reflections and illumination changes. However, it may also erode parts of some vehicles.

2. **Apply only one morphological closing operation with a large kernel:** The large closing reconnects fragmented regions. It helps merge separated car parts into a single detection. No additional morphological operations are applied.



Search space

- α (Alpha) →[1.5-6.0]
- ρ (Rho) →[0.005-0.2]
- Update buffer → [0-4]
- Shadow α →[0.5-0.9]
- Shadow β →[0.8-1.0]
- τs →[20-90]
- τh →[90-140]
- Opening kernel size →[3-7]
- Closing kernel size →[5-21]
- Morph. shape → [ellipse, rectangle]
- Morph. op. → [open, close, open→close, close→open]
- Min area →[200-600]
- Merge distance → [10-50]

Optimal parameters

Category	Parameter	Optimal Value
Background Model	α (Alpha)	5.518
	ρ (Rho)	0.037
	update_buffer	2 px
Shadow Removal	Shadow α	500
	Shadow β	934
	τs,τh	50, 93
Post-Processing	morph_op	Close
	kernel_close_size	11
	morph_shape	Rect
	min_area	583 px²
	merge_dist	12 px

Task 2.1: Adaptive modelling (Team 5)

Hyperparameter Search Space (Optuna)

Category	Parameter	Range
Background Model	α	1.5 – 6.0
	ρ	0.005 – 0.2 (log scale)
	Update buffer	0 – 4 px
Shadow Removal (HSV)	τ_s	20 – 90
	τ_h	90 – 140
	Shadow α	0.5 – 0.9
	Shadow β	0.8 – 1.0

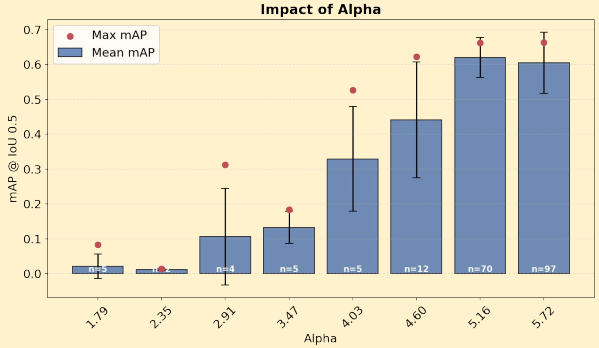
Category	Parameter	Range
Post-Processing	Opening kernel size	3 – 7 (odd)
	Closing kernel size	5 – 21 (odd)
	Morph shape	ellipse / rect
	Morph operation	open / close / open→close / close→open
	Min area	200 – 600 px
	Merge distance	10 – 50 px

Optimal Parameters

Category	Parameter	Optimal Value
Background Model	α (Alpha)	5.518
	ρ (Rho)	0.037
	update_buffer	2 px
Shadow Removal	method	HSV
	Shadow α	500
	Shadow β	934
	τs,τh	50, 93
Post-Processing	morph_op	Close
	kernel_size	11
	morph_shape	Rect
	min_area	583 px²
	merge_dist	12 px

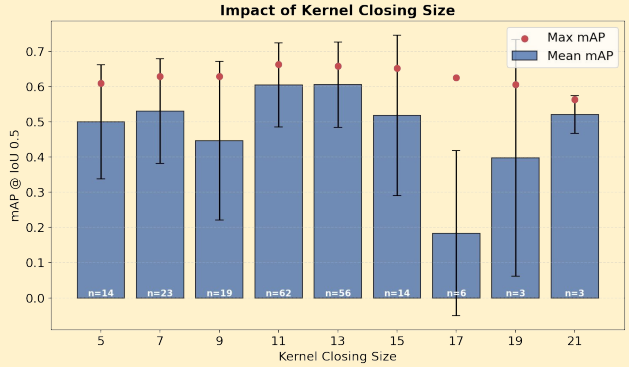
Task 2.1: Adaptive modelling (Team 5)

Impact of alpha



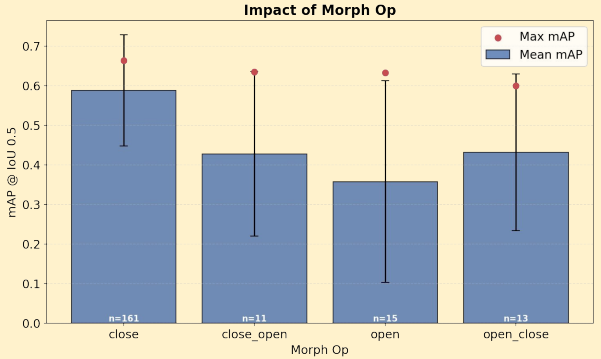
First, we observe that increasing α significantly improves performance. A high threshold is necessary to suppress dynamic background noise, such as waving trees. Although this fragments vehicle detections, the reduction in false positives compensates for it, leading to higher overall mAP.

Impact of kernel size



Second, the kernel size for the closing operation plays a critical role. Because a high α causes fragmentation, a large closing kernel is required to reconnect separated vehicle regions. Smaller kernels (e.g., 3 or 5) fail to bridge these gaps, while larger kernels successfully restore object continuity and improve detection performance.

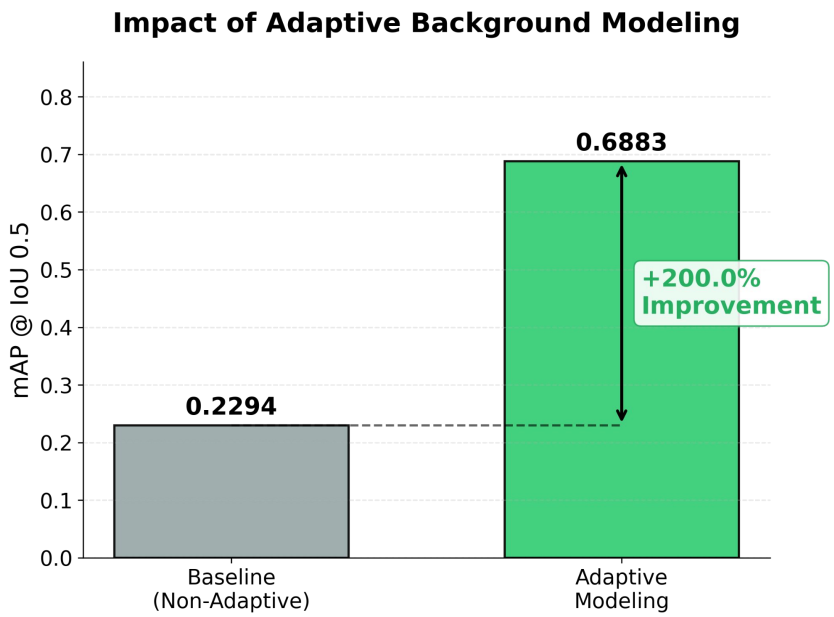
Impact of morphologic operation



Finally, the categorical comparison of morphological operations confirms our hypothesis: the main issue was holes inside detected objects, not noise outside them. Closing clearly outperforms opening, showing that filling internal gaps is more important than removing small external artifacts

SEPARATOR

Task 2.2: Comparison adaptive vs non (Team 5)



Adaptive Advantage

Adaptive background modeling outperforms the static (non-adaptive) approach because it continuously updates the background statistics over time. In real-world traffic scenes, illumination conditions are not constant: the solar angle shifts, clouds pass, shadows move, and tree branches sway. A non-adaptive model can't account for these gradual changes, so it misclassifies background pixels as foreground, which causes a high number of False Positives.

In contrast, the adaptive model updates the background mean using the learning rate ρ, allowing it to follow slow illumination variations while preserving true moving objects. As a result, it maintains a stable background representation throughout the video.

This explains the large improvement in performance: mAP increases from **0.2294 (non-adaptive)** to **0.6883 (adaptive)**, which represents nearly a **200% improvement**.

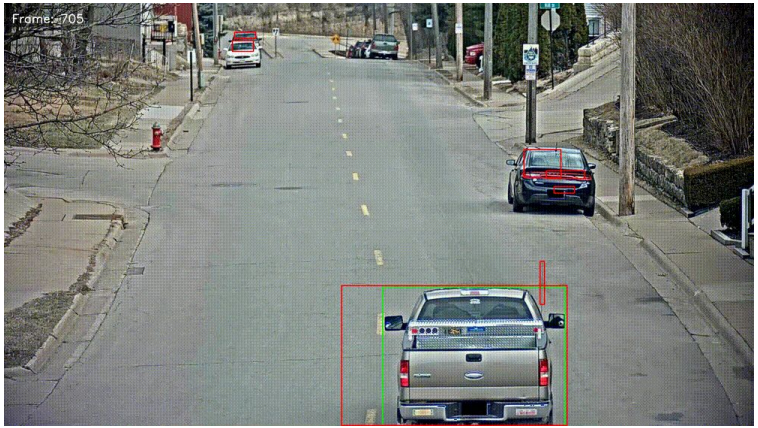
Task 2.2: Comparison adaptive vs non (Team 5)

Background Updating

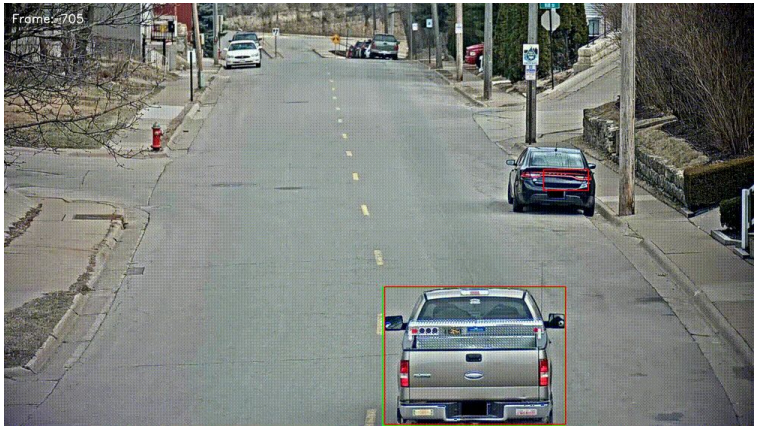
In **Adaptive**, the **learning rate ρ** allows the **background mean (μ) to gradually adapt** to slow illumination changes, such as shifting sunlight or varying cloud cover.

By updating the model over time, the system can track gradual brightness variations without falsely classifying them as foreground.

Non-Adaptive



Adaptive



Impact

By allowing the background model to adapt over time, the False Positive Rate (FPR) drops noticeably in the second half of the video, where illumination changes become more pronounced.

Task 2.2: Comparison adaptive vs non (Team 5)

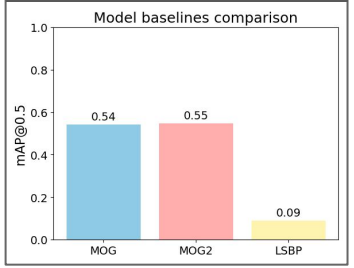
Task 3: Comparison with state-of-the-art

- **Compare with state-of-the-art**
  - P. KaewTraKulPong et.al. *An improved adaptive background mixture model for real-time tracking with shadow detection.* In Video-Based Surveillance Systems, 2002. Implementation: BackgroundSubtractorMOG (OpenCV)
  - Z. Zivkovic et.al. *Efficient adaptive density estimation per image pixel for the task of background subtraction*, Pattern Recognition Letters, 2005. Implementation: BackgroundSubtractorMOG2 (OpenCV)
  - L. Guo, et.al. *Background subtraction using local svd binary pattern*. CVPRW, 2016. Implementation: BackgroundSubtractorLSBP (OpenCV)
  - M. Braham et.al. *Deep background subtraction with scene-specific convolutional neural networks*. In International Conference on Systems, Signals and Image Processing, 2016. No implementation (https://github.com/SaoYan/bgsCNN similar?)
- Evaluate to comment which method (single Gaussian programmed by you or state-of-the-art) performs better

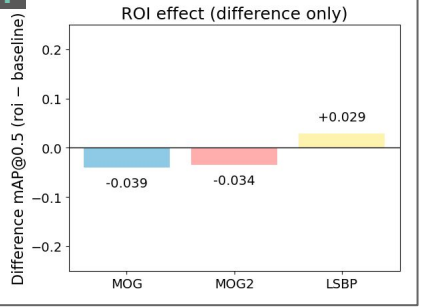
Task 3: Comparison with state-of-the-art (Team 5)

MOG	**Classical fixed-component Gaussian Mixture Model** for pixel-wise background modeling
MOG 2	**Adaptive Gaussian Mixture Model** with **dynamic components** and **shadow detection.**
LSBP	**Local texture-based background** subtraction using **binary patterns** for robustness to illumination changes.

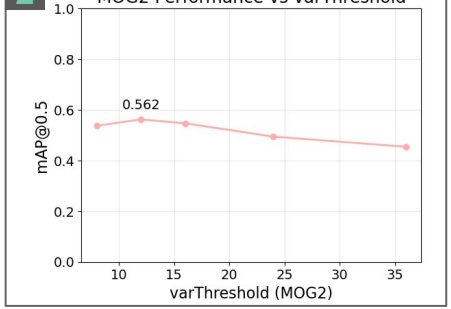
Baselines



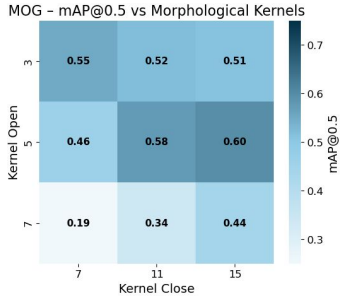
1

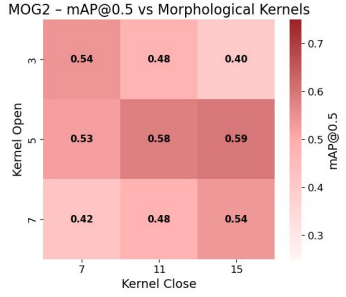


2

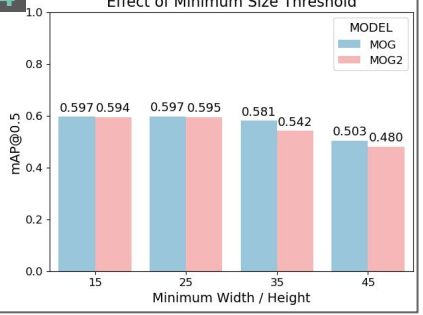


3

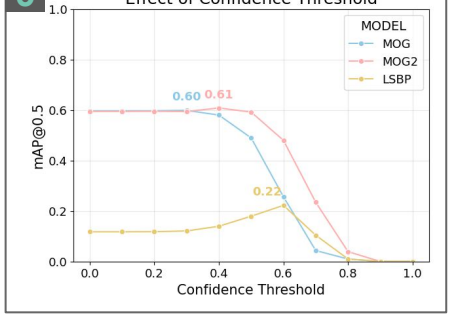




4



5



1. **ROI-based subtraction** decreased mAP in MOG and MOG2, although it slightly improved LSBP. Due to LSBP's high execution time and weak overall performance, we discarded it and removed ROI from the other methods as well.

2. We then tuned **MOG2's key parameter, varThreshold**, which controls foreground sensitivity, and set it to **12** based on performance analysis.

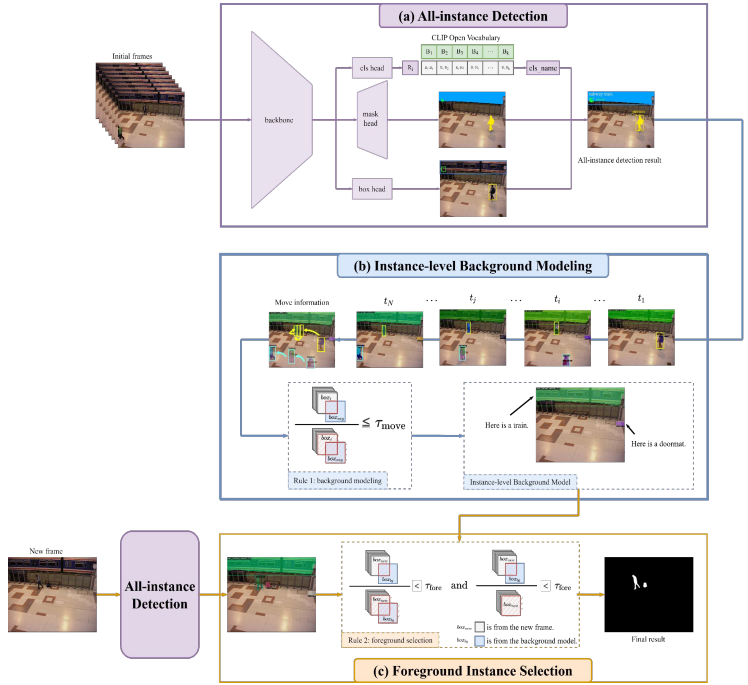
3. Next, we optimized **morphological operations**, finding kernel sizes of **5** (**open**) and **15** (**close**) to be optimal. This increased mAP@50 by about 0.06 in MOG and 0.03 in MOG2.

4. **Bounding box size filtering** showed optimal minimum dimensions around **15-25 pixels**, with little variation in results.

5. Finally, we introduced a **confidence score** based on the **proportion of background pixels inside each box**. Filtering boxes below 0.4 confidence yielded **0.61 mAP@50 in MOG2** and **0.60 in MOG**. Applying a 0.6 threshold to **LSBP** improved it to 0.22, **doubling its initial performance**.

Task 3: Comparison with state-of-the-art (Team 5)

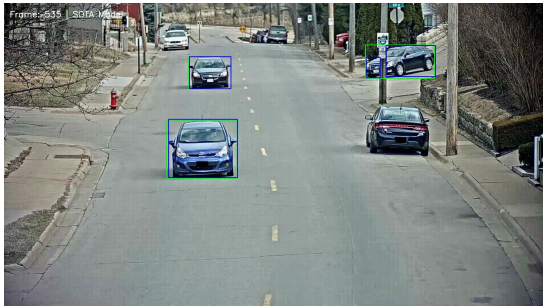
BONUS
ZBS: Zero-shot Background Subtraction [2]



This model utilizes a Swin-B and CLIP architecture to identify objects through semantic knowledge without domain-specific training. Unlike statistical baselines, it is natively immune to shadows and lighting shifts, eliminating the need for manual tuning. We kept the project's original workflow using deep inference as the official repository specifies.

Challenges & Performance

- ➔ **ROI Adjustment**: The semantic model initially detected all vehicles as foreground, including parked cars that the ground truth requires ignoring. We refined the ROI mask to specifically exclude a parked vehicle area, significantly improving performance by aligning the model's semantic detections with Ground Truth constraints.
- ➔ **Detection Constraints**: Metrics were limited by the omission of bicycles and difficulties tracking small, high-speed vehicles in the background.
- ➔ **Sequence Inconsistency**: The model exhibited intermittent detections across frames, a typical Zero-Shot challenge when operating without sequence-specific temporal fine-tuning.
- ➔ **Superior Masking**: Despite lower mAP, the segmentation provides flawless geometric accuracy and native shadow immunity that statistical models lack.
- ➔ **Hyperparameter Tuning**: Systematic optimization of thresholds and area filters would likely enhance performance for this specific sequence.



mAP@0.5 (w/o ROI Adjust.): 0.1604
mAP@0.5 (w/ ROI Adjust.): 0.3397

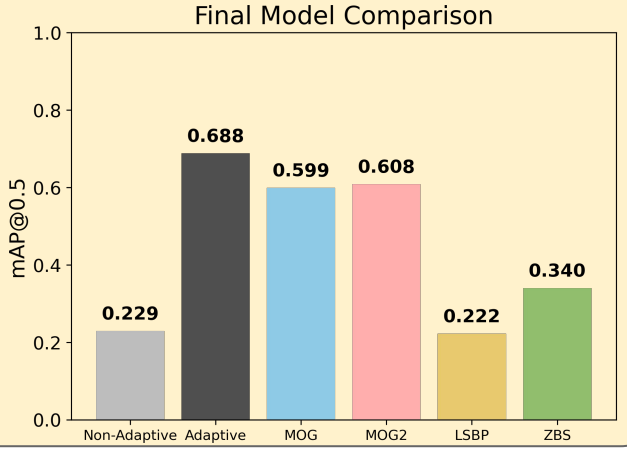
Note: The gifs correspond to ROI Adjust.

[2] Y. An et al., "ZBS: Zero-shot Background Subtraction via instance-level background modeling and foreground selection," in *Proc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023.

Task 3: Comparison with state-of-the-art (Team 5)

Results

- The **Adaptive** (Task 2) model achieves the **highest performance** with mAP@0.5 = **0.688**, outperforming all other approaches.
- The two classical background subtraction methods, **MOG** (**0.599**) and **MOG2** (**0.608**), show very **similar performance**, with only a marginal difference between them.
- Despite being more recent methods, **LSBP** (**0.222**) and **ZBS** (**0.340**) did **not outperform** the classical **MOG-based approaches** in our experiments.
- There is a **substantial performance gap** between the best **Task 1** (**Non-Adaptive**, **0.229**) and the best **Task 2** (**Adaptive**, **0.688**) model, highlighting the significant improvement obtained when using an **adaptive background modeling strategy**.



Qualitative Results

While **ZBS** obtains a lower mAP@0.5, it **produces cleaner** and **more precise segmentation masks**, as illustrated in the GIFs above.

Conclusions

- **Adaptive background modeling** clearly **outperforms** all other methods, achieving the highest mAP@0.5 (**0.688**).
- There is a **large performance gap** between **non-adaptive** and **adaptive models**, confirming that background adaptation is crucial in dynamic scenes.
- The **main improvement** comes from the model's ability to **handle illumination changes** and long-term scene variations.
- Although **ZBS** has lower quantitative performance, it produces **visually cleaner segmentation masks**, highlighting that mAP does not fully capture qualitative quality.
- The bad results of **LSBP and ZBS** are due to the lack of hyperparameter search to find the most optimal values.